

GERÊNCIA DE REDES

CI/CD na Prática

com GitHub Actions e Render



LABORATÓRIO 6

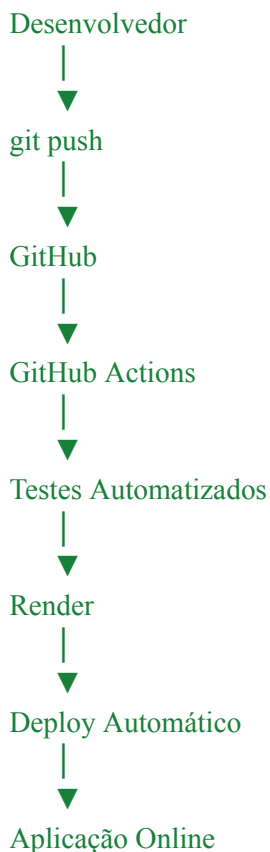
CI/CD na Prática com GitHub Actions e Render

Agora vamos trabalhar com CI/CD na prática usando GitHub Actions e Render. Os conceitos e ferramentas trabalhadas neste lab são bastante usados(as) nos ambientes de devops modernos



OBJETIVO DO LAB

Construir uma pipeline completa de Integração Contínua e Entrega Contínua (CI/CD) para uma aplicação web Node.js.





CONCEITO CENTRAL

O que é CI/CD?

CI – Continuous Integration

Integração Contínua é a prática de validar automaticamente o código sempre que uma alteração é enviada ao repositório. Exemplo:

```
None
Programador
  ↓
Git Push
  ↓
Testes Automáticos
  ↓
Aprovado ou Reprovado
```

Benefícios:

- *Detecta erros rapidamente*
- *Evita código quebrado*
- *Mantém qualidade*

CD – Continuous Delivery / Deployment

Após a aprovação dos testes, o sistema realiza automaticamente a publicação da nova versão.

```
None
Teste
  ↓
Build
  ↓
Deploy
  ↓
Aplicação Atualizada
```

Benefícios:

- *Menos erros humanos*
- *Atualizações rápidas*
- *Maior produtividade*

Estrutura Geral do Projeto

Antes de criar qualquer arquivo, organize o projeto da seguinte forma:

None

```
ci-cd-lab/
|
├─ app.js
├─ package.json
├─ Dockerfile
├─ .gitignore
|
├─ tests/
│   └─ app.test.js
|
└─ .github/
    └─ workflows/
        └─ ci.yml
```

O que significa cada item?

Item	Finalidade
app.js	Aplicação principal
package.json	Dependências e scripts do Node
Dockerfile	Instruções para construir a imagem Docker
tests	Testes automatizados
.github/workflows	Pipelines do GitHub Actions
ci.yml	Definição da pipeline CI/CD

Parte 1 – Continuous Integration (CI)

Passo 1 – Criar o Projeto

Crie a pasta:

None

```
mkdir ci-cd-lab  
  
cd ci-cd-lab
```

Passo 2 – Inicializar Projeto Node.js

Execute:

None

```
npm init -y
```

Passo 3 – Instalar Dependências

Aplicação:

None

```
npm install express
```

Testes:

None

```
npm install --save-dev jest supertest
```

Explicação

jest

Framework de testes.

Permite validar automaticamente o funcionamento da aplicação.

supertest

Biblioteca que simula requisições HTTP.

Permite testar endpoints sem abrir navegador.

Passo 4 – Criar a Aplicação

Arquivo:

None

app.js

None

```
const express = require("express");

const app = express();

app.use(express.json());

/*
 * Banco de dados em memória
 */
let usuarios = [
  { id: 1, nome: "Marcelo" }
];

/*
 * Rota principal
 */
app.get("/", (req, res) => {
```

```

    res.status(200).send("Aplicação funcionando!");
  });

  /*
   * Health Check
   */
  app.get("/health", (req, res) => {
    res.status(200).json({
      status: "UP"
    });
  });

  /*
   * Listar usuários
   */
  app.get("/usuarios", (req, res) => {
    res.json(usuarios);
  });

  /*
   * Buscar usuário por ID
   */
  app.get("/usuarios/:id", (req, res) => {

    const usuario = usuarios.find(
      u => u.id === parseInt(req.params.id)
    );

    if (!usuario) {
      return res.status(404).json({
        erro: "Usuário não encontrado"
      });
    }

    res.json(usuario);
  });

  /*
   * Criar usuário
   */
  app.post("/usuarios", (req, res) => {

```

```

    const { nome } = req.body;

    if (!nome) {
      return res.status(400).json({
        erro: "Nome é obrigatório"
      });
    }

    const novoUsuario = {
      id: usuarios.length + 1,
      nome
    };

    usuarios.push(novoUsuario);

    res.status(201).json(novoUsuario);
  });

  /*
   * Executa somente em produção
   */

  const PORT = process.env.PORT || 3000;
  if (require.main === module) {
    app.listen(PORT, () => {
      console.log(`Servidor iniciado na porta ${PORT}`);
    });
  }

  /*
   * Exporta para testes
   */
  module.exports = app;

```

Explicação do código

process.env.PORT

None

```
process.env.PORT
```

Obtém a porta definida pelo ambiente.

O Render utiliza portas dinâmicas.

Caso não exista variável PORT, usa:

None

```
3000
```

Passo 5 – Criar os Testes

Pasta:

None

```
tests/
```

Arquivo:

None

```
tests/app.test.js
```

None

```
const request = require("supertest");
const app = require("../app");

describe("API de Usuários", () => {

  test("GET / deve retornar 200", async () => {
```

```

    const response = await request(app)
      .get("/");

    expect(response.statusCode).toBe(200);
  });

test("GET /health deve retornar UP", async () => {

  const response = await request(app)
    .get("/health");

  expect(response.statusCode).toBe(200);

  expect(response.body.status)
    .toBe("UP");
});

test("GET /usuarios deve retornar lista", async () => {

  const response = await request(app)
    .get("/usuarios");

  expect(response.statusCode).toBe(200);

  expect(Array.isArray(response.body))
    .toBe(true);
});

test("GET usuário existente", async () => {

  const response = await request(app)
    .get("/usuarios/1");

  expect(response.statusCode).toBe(200);

  expect(response.body.nome)
    .toBe("Marcelo");
});

test("GET usuário inexistente", async () => {

  const response = await request(app)

```

```
        .get("/usuarios/999");

        expect(response.statusCode).toBe(404);
    });

    test("POST criar usuário", async () => {

        const response = await request(app)
            .post("/usuarios")
            .send({
                nome: "Maria"
            });

        expect(response.statusCode).toBe(201);

        expect(response.body.nome)
            .toBe("Maria");
    });

    test("POST sem nome deve falhar", async () => {

        const response = await request(app)
            .post("/usuarios")
            .send({});

        expect(response.statusCode).toBe(400);
    });

});
```

Explicação

describe()

Agrupa testes relacionados.

test()

Define um teste.

expect()

Verifica o resultado esperado.

toBe(200)

Confirma que a aplicação respondeu:

None

HTTP 200 OK

Passo 6 – Atualizar package.json

Substitua:

None

```
"scripts": {  
  "test": "jest",  
  "test:coverage": "jest --coverage"  
}
```

Isso irá nos permitir executar comandos simplificados:

None

npm test

npm test:coverage

Passo 7 – Testar Localmente

Executar:

None

```
npm test
```

Resultado esperado:

None

```
PASS tests/app.test.js
  API de Usuários
    ✓ GET / deve retornar 200 (23 ms)
    ✓ GET /health deve retornar UP (3 ms)
    ✓ GET /usuarios deve retornar lista (4 ms)
    ✓ GET usuário existente (4 ms)
    ✓ GET usuário inexistente (8 ms)
    ✓ POST criar usuário (24 ms)
    ✓ POST sem nome deve falhar (3 ms)

Test Suites: 1 passed, 1 total
Tests:       7 passed, 7 total
Snapshots:   0 total
Time:        0.512 s
Ran all test suites.
```

Passo 8 – Criar Dockerfile

Arquivo:

None

```
Dockerfile
```

None

```
FROM node:22
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm install --silent
```

```
COPY . .
```

```
EXPOSE 3000
```

```
CMD ["node", "app.js"]
```

Explicação Linha por Linha

FROM

None

```
FROM node:22
```

Imagem base.

Contém:

- Node.js 22
- npm

WORKDIR

None

```
WORKDIR /app
```

Diretório de trabalho dentro do container.

COPY

None

```
COPY package*.json ./
```

Copia:

None

```
package.json
```

```
package-lock.json
```

RUN

None

```
RUN npm install
```

Instala dependências.

EXPOSE

None

```
EXPOSE 3000
```

Registra a porta usada.

CMD

None

```
CMD ["node", "app.js"]
```

Comando executado quando o container inicia.

Passo 9 – Construir Imagem Docker

None

```
docker build -t ci-cd-lab .
```

Explicação

docker build

Constrói imagem.

-t

Tag da imagem.

None

`ci-cd-lab`

nome atribuído.

.

Contexto atual.

Passo 10 – Executar Container

None

`docker run -d -p 3000:3000 ci-cd-lab`

Explicação

-d

Detached mode.

Executa em segundo plano.

-p

Mapeamento de portas.

None

`HOST:CONTAINER`

None

```
3000:3000
```

Passo 11 – Criar .gitignore

Crie o arquivo:

None

```
.gitignore
```

Insira o node_modules

None

```
node_modules
```

Passo 12 – Criar Repositório Git

Crie um repositório chamado:

None

```
ci-cd-lab
```

Agora no diretório do projeto, vamos trackear o versionamento

None

```
git init
git add .
git commit -m "Projeto inicial"
git branch -M main
git remote add origin https://github.com/douglasnunesif/ci-cd-lab.git
git push -u origin main
```

Passo 13 – Criar Pipeline GitHub Actions

O que é?

GitHub Actions é a plataforma de automação do GitHub.

Ela permite executar:

- testes
- builds
- deploys

automaticamente.

Arquivo:

None

`.github/workflows/ci.yml`

None

```
name: CI Pipeline

on:
  push:
    branches:
      - main

  pull_request:
    branches:
      - main

jobs:

  test:

    runs-on: ubuntu-latest

    steps:

      - name: Checkout
        uses: actions/checkout@v5
```

```
- name: Configurar Node.js
  uses: actions/setup-node@v5

  with:
    node-version: 24

- name: Instalar dependências
  run: npm install

- name: Executar testes
  run: npm test

- name: Gerar cobertura
  run: npm run test:coverage

- name: Construir imagem Docker
  run: docker build -t ci-cd-lab .
```

O que essa pipeline faz?

Sempre que ocorrer:

```
None
git push
```

o GitHub irá:

1. Criar uma máquina virtual temporária
2. Baixar o projeto
3. Instalar Node.js
4. Instalar dependências
5. Executar os testes

Explicação da Pipeline

on

None

`on:`

`push:`

Executa quando ocorre:

None

`git push`

branches

None

`main`

Executa somente na branch principal.

jobs

Conjunto de tarefas.

runs-on

None

`ubuntu-latest`

Máquina virtual criada pelo GitHub.

actions/checkout@v5

Baixa o código do repositório.

actions/setup-node@v5

Instala Node.js.

node-version

None

22

Versão utilizada pela pipeline.

run

Executa comandos Linux.

Exemplo:

None

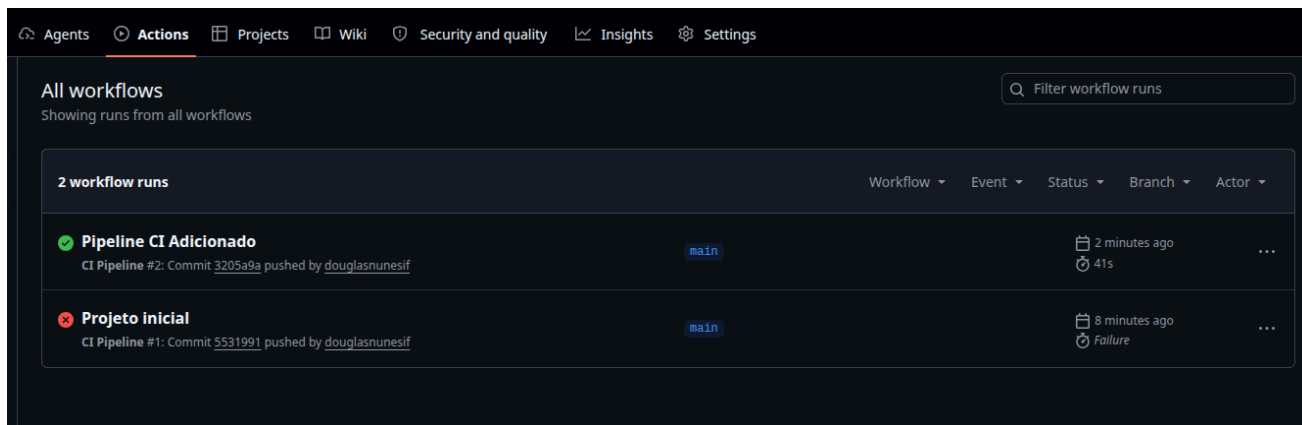
run: npm test

Passo 14 – Enviar para GitHub

None

```
git add .  
  
git commit -m "Pipeline CI Adicionado"  
  
git push origin main
```

Acesse a aba **Action** do repositório para verificar o processo de automação acontecendo



Fluxo Completo da Pipeline

None

Desenvolvedor



git push



GitHub



GitHub Actions



Instala Node.js 24



npm install



npm test



Pipeline Verde



Pronto para Deploy

Parte 2 – Continuous Deployment (CD) com Render

Após concluir a etapa de Integração Contínua (CI) com GitHub Actions, vamos implementar o **Continuous Deployment (CD)**.

O objetivo é que toda alteração enviada para a branch `main` seja automaticamente publicada na Internet, sem necessidade de acessar servidores manualmente.

O fluxo completo ficará assim:



O que é o Render?

O Render é uma plataforma PaaS (Platform as a Service).

Ele funciona de forma semelhante ao:

- Heroku
- Railway
- Fly.io
- Azure App Service
- AWS Elastic Beanstalk

A principal vantagem para fins educacionais é que possui um plano gratuito suficiente para laboratórios de CI/CD.

O Render realiza automaticamente:

- ✓ Build da aplicação
 - ✓ Deploy
 - ✓ Provisionamento do servidor
 - ✓ HTTPS
 - ✓ Integração com GitHub
-

Arquitetura Final



Passo 15 – Criar Conta no Render

Acesse:

[Render](#)

Clique em:

None

Get Started

Passo 16 – Login com GitHub

Selecione:

None

Continue with GitHub

O que acontece aqui?

O Render solicitará autorização para acessar seus repositórios GitHub.

Isso permitirá:

None

Ler código-fonte

Detectar alterações

Executar deploy automático

Passo 17 – Autorizar o Repositório

Durante a autorização, escolha:

None
Only select repositories

e selecione:

None
ci-cd-lab

Por que fazer isso?

Boa prática de segurança.

Em produção, nunca concedemos acesso irrestrito a todos os repositórios.

Passo 18 – Criar o Serviço

No painel do Render:

None
New +

Escolha:

None
Web Service

Passo 19 – Selecionar Repositório

Selecione:

None
ci-cd-lab

Se você não conectou a sua conta pelo Github, poderá informar o link do repositório no campo do formulário que for apresentado, em **Source code**

None
<https://github.com/douglasnunesif/ci-cd-lab.git>

Passo 20 – Configurar o Serviço

Preencha:

Nome

None
ci-cd-lab

Região

Escolha uma ou deixe da forma com foi sugerida:

None
Oregon

Branch

None
main

Passo 21 – Escolher Plano

Selecione:

None
Free

O que o plano gratuito oferece?

- Deploy automático
- HTTPS
- Integração GitHub
- Aplicação online

Perfeito para o nosso laboratório

Passo 22 – Criar Serviço

Clique:

None
Create Web Service

O que acontece agora?

O Render executará:

```
None
git clone
npm install
node app.js
```

automaticamente.

Passo 23 – Acompanhar Build

Você verá algo semelhante a:

None

Cloning repository

Depois:

None

Installing dependencies

Depois:

None

Deploying...

Finalmente:

None

Live

Passo 24 – Testar Aplicação

O Render fornecerá uma URL semelhante:

None

<https://ci-cd-lab.onrender.com>

Acesse.

Resultado esperado:

None

Laboratório CI/CD funcionando!

Passo 25 – Ajustar Aplicação para Produção

Verifique se o arquivo:

```
None  
app.js
```

contém:

```
None  
const PORT = process.env.PORT || 3000;
```

Por que isso é obrigatório?

O Render define dinamicamente a porta.

Exemplo:

```
None  
10000
```

ou

```
None  
12045
```

ou qualquer outra.

Se você fixar:

```
None  
3000
```

o deploy falhará.

Passo 26 – Habilitar Deploy Automático

No Render:

None
Settings

Verifique:

None
Auto-Deploy => On Commit

O que isso faz?

Sempre que ocorrer:

None
git push

o Render executará:

None
Novo Build
Novo Deploy

automaticamente.

PARTE 27 – Integrando GitHub Actions e Render

Agora vamos garantir:

None
Teste → Deploy

Dentro de Settings, no Render:

None

Settings

Copiar:

None

Deploy Hook URL

Criar Secret no repositório do GitHub

Repositório

None

Settings



None

Secrets and Variables



None

Actions



None

New Repository Secret

Nome:

None

RENDER_DEPLOY_HOOK

Valor:

None

URL copiada do Render

Atualizar Pipeline

None

name: CI Pipeline

on:

push:

branches:

- main

pull_request:

branches:

- main

jobs:

test:

runs-on: ubuntu-latest

steps:

- name: Checkout

uses: actions/checkout@v5

- name: Configurar Node.js

uses: actions/setup-node@v5

with:

node-version: 24

- name: Instalar dependências

run: npm install

- name: Executar testes

run: npm test

- name: Gerar cobertura

run: npm run test:coverage

- name: Construir imagem Docker

run: docker build -t ci-cd-lab .

deploy:

needs: test

runs-on: ubuntu-latest

```
steps:
  - name: Disparar Deploy no Render
    run: |
      curl -X POST \
        ${{ secrets.RENDER_DEPLOY_HOOK }}
```

O que acontece agora?

Fluxo completo:

```
None
Ao alterar o código
    ↓
git add .
git commit
git push
    ↓
GitHub Actions
    ↓
Executa Testes
    ↓
Sucesso
    ↓
Render Deploy Hook
    ↓
Build
    ↓
Deploy
    ↓
Aplicação Atualizada
```

Passo 28 – Testar o Deploy Automático

Altere:

None

```
res.send("Aplicação funcionando!");
```

para:

None

```
res.send("Laboratório CI/CD funcionando! Nova versão implantada automaticamente!");
```

Faça:

None

```
git add .  
  
git commit -m "Atualização da página inicial"  
  
git push
```

O que acontecerá?

GitHub:

None

```
Recebe o código
```

↓

GitHub Actions:

None

```
Executa testes
```

↓

Render:

None

Detecta alteração



Render:

None

Executa build



Render:

None

Publica nova versão



Internet:

None

Nova versão disponível

Passo 29 – Visualizar Logs do Deploy

No painel do Render:

None

Logs

Você poderá acompanhar:

None

Build Logs

e

None

Runtime Logs

O que são Build Logs?

Mostram:

None

npm install

None

node_modules

None

compilação

O que são Runtime Logs?

Mostram:

None

console.log()

None

erros

None

requisições

Passo 30 – Simular Falha de Deploy

Modifique no `app.test.js`, no teste da rota padrão GET `/`:

```
None  
expect(response.statusCode).toBe(404);
```

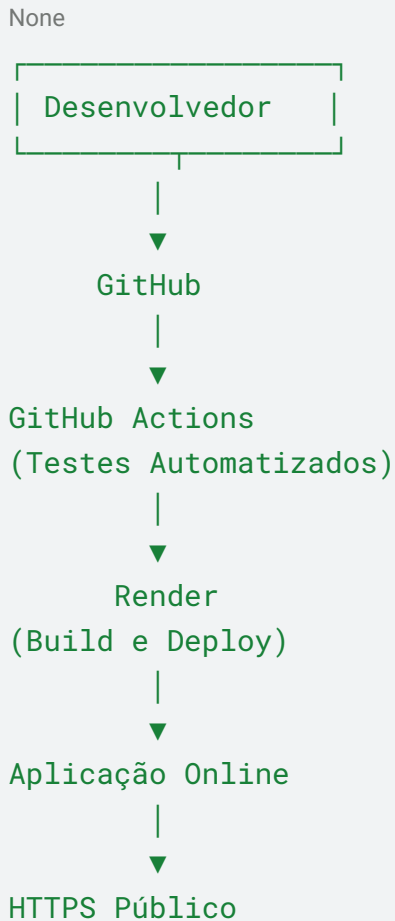
(deixando um erro proposital)

Faça:

```
None  
git push
```

Resultado Final do Laboratório

Ao concluir esta etapa, os alunos terão construído uma pipeline real de DevOps:



Conceitos praticados

- ✓ Git
- ✓ GitHub
- ✓ Docker
- ✓ Node.js
- ✓ Testes Automatizados (Jest)
- ✓ GitHub Actions
- ✓ CI (Continuous Integration)
- ✓ CD (Continuous Deployment)
- ✓ Cloud Computing
- ✓ Deploy em Produção